LOOP Engine

1. LOOP Engine – Kompakte System-Referenz für KI-Modelle
2. Grundsyntax
3. Referenz der LOOP-Parameter
4. Condition im LOOP
5. WHILE
6. Wann WHILE verwenden?
7. DO_WHILE
8. Wann DO_WHILE verwenden?
9. SQL Polling – vollständiges Beispiel
10. Ablauf des SQL-Polling-Beispiels
11. REST Polling
12. Mehrere aktive Status
13. Mehrere Endzustände
14. CONTAINS-Polling
15. Warum ENGINE → BUFFER → Condition?
16. maxIterations
17. timeoutMs
18. maxIterations und timeoutMs zusammen
19. WAIT und LOOP-Timeout
20. Wichtige DO_WHILE-Semantik
21. Kein BREAK-Step
22. LOOP Logging
23. LOOP vs. WAIT
24. Troubleshooting
    - LOOP endet nach einer Iteration
    - LOOP erreicht maxIterations
    - LOOP erreicht timeoutMs
    - Buffer nicht gefunden
    - Condition-Wert existiert vor WHILE nicht
25. Best Practices
    - DO_WHILE für Polling
    - Technische Antworten in Buffer überführen
    - Polling-Intervall sinnvoll wählen
    - Immer Safety Guards definieren
26. Kurzreferenz für KI / RAG

Die LOOP-Engine ermöglicht die wiederholte Ausführung einer Gruppe von AGATE-Schritten.

Die Wiederholung wird über eine `condition` gesteuert und zusätzlich durch `maxIterations` und `timeoutMs` abgesichert.

Typische Anwendungsfälle:

- Polling
- Warten auf asynchrone Backend-Verarbeitung
- wiederholte Statusabfragen
- wiederholte SQL-Abfragen
- API-Status-Polling

- technische Synchronisation zwischen Testschritten

---

1. LOOP Engine – Kompakte System-Referenz für KI-Modelle

```
AGATE ENGINE: LOOP

PURPOSE:
Repeat nested AGATE steps while a condition remains true.

TYPE:
LOOP

NO op FIELD.

REQUIRED:
mode
condition
maxIterations
timeoutMs
steps

MODES:

WHILE
- condition before body
- 0..N iterations

DO_WHILE
- body before condition
- 1..N iterations
- preferred when condition value is generated inside the LOOP

CONDITION:

LOOP uses the general AGATE Condition Handling syntax.

LOOP-specific semantics:

true
-> execute another iteration

false
-> terminate LOOP successfully

SAFETY:

maxIterations
-> maximum number of iterations

timeoutMs
-> maximum total runtime of the complete LOOP

If a safety guard is reached and condition still requires another
iteration:
LOOP = FAILED

RECOMMENDED POLLING:

ENGINE EXEC
-> ENGINE BUFFER
-> WAIT
-> LOOP condition

NO BREAK STEP.

DO_WHILE executes the complete child-step list before checking
condition.

```

```
IMPORTANT:
The full Condition syntax is documented separately in:
Condition Handling – AGATE DSL
```

2. Grundsyntax

```yaml
- type: LOOP
  mode: DO_WHILE
  condition: "{B[status]} != 'READY'"
  maxIterations: 20
  timeoutMs: 60000

  steps:

    - type: REST
      op: EXEC
      command: demo.status
      response: status_response

    - type: WAIT
      value: "2000"
```

3. Referenz der LOOP-Parameter

Parameter	Beschreibung	Erforderlich
`type`	Muss `LOOP` sein	Ja
`mode`	`WHILE` oder `DO_WHILE`	Ja
`condition`	Fortsetzungsbedingung des LOOPs	Ja
`maxIterations`	Maximale Anzahl Iterationen	Ja
`timeoutMs`	Maximale Gesamtdauer des LOOPs in Millisekunden	Ja
`steps`	Child-Steps einer Iteration	Ja
`id`	Optionaler Step-Identifier	Nein

4. Condition im LOOP

LOOP verwendet die allgemeine AGATE Condition-Syntax.

Beispiele:

```
condition: "{B[count]} == 0"
```

```
condition: "{B[status]} != 'READY'"
```

```
condition: "{B[status]} == 'PENDING' OR {B[status]} == 'RUNNING'"
```

```
condition: "NOT ({B[status]} == 'DONE' OR {B[status]} == 'SUCCESS')"
```

Die vollständige Operator- und Syntax-Referenz befindet sich im separaten Dokument:

Condition Handling – AGATE DSL

Für LOOP ist entscheidend:

```
condition true
-> nächste Iteration

condition false
-> LOOP erfolgreich beenden
```

---

5. WHILE

Bei `WHILE` wird die Condition **vor jeder Iteration** geprüft.

```
- type: LOOP
  mode: WHILE
  condition: "{B[continue_processing]} == 'true'"
  maxIterations: 10
  timeoutMs: 30000

  steps:

    - type: CMD
      op: EXEC
      command: "echo processing"
```

Ablauf:

```
Condition
    |
    ├── false
    |     ↓
    |   LOOP Ende
    |
    └── true
          ↓
       Child-Steps
          ↓
       Condition
```

Ein WHILE-LOOP kann **0 Iterationen** ausführen.

---

6. Wann WHILE verwenden?

WHILE ist sinnvoll, wenn der Condition-Wert **bereits vor dem LOOP existiert**.

Beispiel:

```yaml
- type: BUFFER
  op: EXEC
  name: continue_processing
  value: "true"

- type: LOOP
  mode: WHILE
  condition: "{B[continue_processing]} == 'true'"
  maxIterations: 10
  timeoutMs: 30000

  steps:
    ...
```

Hier existiert:

```
B[continue_processing]
```

bereits vor der ersten LOOP-Auswertung.

---

7. DO_WHILE

Bei `DO_WHILE` wird zuerst der komplette LOOP-Body ausgeführt.

Danach wird die Condition geprüft.

```yaml
- type: LOOP
  mode: DO_WHILE
  condition: "{B[result_count]} == 0"
  maxIterations: 30
  timeoutMs: 130000

  steps:

    - type: SQL
      op: EXEC
      command: "SELECT ..."
      response: result

    - type: SQL
      op: BUFFER
      response: result
      row: "0"
      column: "0"
      name: result_count

    - type: WAIT
      value: "4000"
```

Ablauf:

```
Child-Steps
     │
     ▼
Condition
     │
     ├── true
     │     ↓
     │ nächste Iteration
     │
     └── false
           ↓
        LOOP Ende
```

Ein DO_WHILE führt mindestens **eine Iteration** aus.

---

8. Wann DO_WHILE verwenden?

DO_WHILE ist der bevorzugte Modus für Polling, wenn der Condition-Wert erst innerhalb des LOOPs erzeugt wird.

Typisches Muster:

```
SQL
 ↓
BUFFER
 ↓
Condition
```

Vor der ersten Iteration existiert der Buffer noch nicht.

Daher:

```
mode: DO_WHILE
```

---

9. SQL Polling – vollständiges Beispiel

```yaml
- id: wait_until_job_completed
  type: LOOP
  mode: DO_WHILE
  condition: "{B[job_completed_count]} == 0"
  maxIterations: 30
  timeoutMs: 130000

  steps:

    - type: SQL
      op: EXEC
      command: >
        SELECT COUNT(*)
        FROM DEMO_JOB_STATUS
        WHERE JOB_ID = '{R[JobId]}'
          AND STATUS = 'COMPLETED'
      response: job_status_response

    - type: SQL
      op: BUFFER
      response: job_status_response
      row: "0"
      column: "0"
      name: job_completed_count

    - type: WAIT
      value: "4000"
```

---

10. Ablauf des SQL-Polling-Beispiels

Iteration 1:

```
SQL -> count 0
BUFFER -> job_completed_count = 0
WAIT 4000
Condition: 0 == 0 -> true
```

Weitere Iteration.

Iteration 2:

```
SQL -> count 0
BUFFER -> 0
WAIT
Condition -> true
```

Weitere Iteration.

Spätere Iteration:

```
SQL -> count 1
BUFFER -> 1
WAIT
Condition: 1 == 0 -> false
```

LOOP Ende.

---

11. REST Polling

```
- type: LOOP
  mode: DO_WHILE
  condition: "{B[api_status]} != 'READY'"
  maxIterations: 20
  timeoutMs: 60000

  steps:

    - type: REST
      op: EXEC
      command: demo.status
      response: status_response

    # Status über die passende BUFFER-Funktion
    # nach B[api_status] übernehmen

    - type: WAIT
      value: "2000"
```

Möglicher Verlauf:

```
PENDING
-> weiter

PROCESSING
-> weiter

READY
-> LOOP Ende
```

---

12. Mehrere aktive Status

```
- type: LOOP
  mode: DO_WHILE
  condition: "{B[status]} == 'PENDING' OR {B[status]} == 'RUNNING'"
  maxIterations: 20
  timeoutMs: 60000

  steps:
    ...
```

Der LOOP läuft bei:

```
PENDING
RUNNING
```

weiter.

Bei beispielsweise:

```
DONE
READY
FAILED
```

ist die Condition false und der LOOP endet.

---

13. Mehrere Endzustände

```
- type: LOOP
  mode: DO_WHILE
  condition: "NOT ({B[status]} == 'DONE' OR {B[status]} == 'SUCCESS')"
  maxIterations: 20
  timeoutMs: 60000

  steps:
    ...
```

Der LOOP endet bei:

```
DONE
SUCCESS
```

---

14. CONTAINS-Polling

```
- type: LOOP
  mode: DO_WHILE
  condition: "NOT {B[log_output]} CONTAINS 'PROCESS FINISHED'"
  maxIterations: 20
  timeoutMs: 60000

  steps:

    - type: CMD
      op: EXEC
      command: "demo-status-command"

    - type: WAIT
      value: "2000"
```

Der LOOP endet, sobald der erwartete Text vorhanden ist.

---

15. Warum ENGINE → BUFFER → Condition?

Empfohlen:

```
ENGINE
   |
   ▼
RESPONSE
   |
   ▼
```

```
BUFFER
   │
   ▼
CONDITION
```

Beispiel:

```yaml
- type: SQL
  op: EXEC
  response: result

- type: SQL
  op: BUFFER
  response: result
  row: "0"
  column: "0"
  name: result_count
```

Dann:

```
condition: "{B[result_count]} == 0"
```

Damit bleibt LOOP unabhängig von der technischen Quelle des Werts.

Der Wert kann aus:

```
SQL
REST
SOAP
CMD
OC
FILE
...
```

stammen.

---

16. maxIterations

`maxIterations` schützt vor Endlosschleifen.

```
maxIterations: 30
```

Der LOOP darf maximal 30 Iterationen ausführen.

Wenn die Condition danach immer noch eine weitere Iteration verlangt, schlägt der LOOP fehl.

`maxIterations` ist ein Safety Guard und kein regulärer Exit.

---

17. timeoutMs

`timeoutMs` begrenzt die maximale Gesamtdauer des LOOPs.

```
timeoutMs: 130000
```

Die Zeit umfasst:

```
alle Iterationen
+
SQL
```

```
+
REST
+
SOAP
+
CMD
+
WAIT
+
CALL
+
weitere Child-Steps
```

---

18. maxIterations und timeoutMs zusammen

Beide Parameter schützen gegen unterschiedliche Fehler.

`maxIterations`:

```
Schutz gegen sehr schnelle Endlosschleifen
```

`timeoutMs`:

```
Schutz gegen sehr lange LOOP-Ausführungen
```

Beide sollen bei jedem LOOP definiert sein.

---

19. WAIT und LOOP-Timeout

Beispiel:

```
maxIterations: 30
timeoutMs: 130000
```

mit:

```
- type: WAIT
  value: "4000"
```

Maximale reine WAIT-Zeit:

```
30 × 4000 ms
=
120000 ms
```

Zusätzlich benötigen:

```
SQL
BUFFER
Condition
Logging
```

Zeit.

Daher ist:

```
timeoutMs: 130000
```

sinnvoller als exakt:

```
timeoutMs: 120000
```

---

20. Wichtige DO_WHILE-Semantik

Bei DO_WHILE wird der **gesamte Child-Step-Block** ausgeführt, bevor die Condition geprüft wird.

Beispiel:

```
steps:

  - type: SQL
    ...

  - type: SQL
    op: BUFFER
    ...

  - type: WAIT
    value: "4000"
```

Wenn der SQL-Step bereits den Zielzustand findet:

```
SQL -> 1
BUFFER -> 1
```

wird trotzdem:

```
WAIT 4000
```

ausgeführt.

Erst danach:

```
Condition -> false
LOOP Ende
```

---

21. Kein BREAK-Step

Die aktuelle LOOP-Version besitzt keinen separaten:

```
- type: BREAK
```

Step.

Die Steuerung erfolgt ausschließlich über:

```
mode
+
condition
```

Beispiel einer BREAK-Denkweise:

```
BREAK wenn count > 0
```

wird als Fortsetzungsbedingung modelliert:

```
condition: "{B[count]} == 0"
```

Solange `count == 0` gilt, läuft der LOOP.

Sobald `count != 0` gilt, endet er.

---

22. LOOP Logging

Typischer Start:

```
>>> LOOP      : mode=DO_WHILE | maxIterations=30 | timeoutMs=120000
>>> LOOP ITER : 1/30
```

Nach einer Iteration:

```
>>> LOOP COND : {B[result_count]} == 0 -> true
>>> LOOP ITER : 2/30
```

Beim regulären Ende:

```
>>> LOOP COND : {B[result_count]} == 0 -> false
>>> LOOP END  : iterations=4 | duration=16082 ms
```

---

23. LOOP vs. WAIT

WAIT	LOOP
wartet feste Dauer	wartet auf einen Zustand
keine Wiederholung	wiederholt Child-Steps
keine Condition	Condition-gesteuert
kann unnötig lange warten	kann frühzeitig enden
kann zu kurz sein	prüft tatsächlichen Zustand

Statisch:

```
- type: WAIT
  value: "15000"
```

bedeutet:

```
Immer 15 Sekunden warten.
```

LOOP:

```
- type: LOOP
  mode: DO_WHILE
  condition: "{B[ready]} != 'true'"
```

bedeutet:

```
Wiederholt prüfen und erst weitergehen,
wenn ready == true.
```

24. Troubleshooting

LOOP endet nach einer Iteration

Prüfen:

```
>>> LOOP COND ... -> false
```

Wenn die Condition `false` ergibt, endet der LOOP korrekt.

LOOP erreicht maxIterations

Prüfen:

- Wird der Buffer aktualisiert?
- Ist der richtige Buffer referenziert?
- Wird der erwartete Zustand überhaupt erreicht?
- Ist die Condition korrekt formuliert?

LOOP erreicht timeoutMs

Prüfen:

- Polling-Intervall
- Child-Step-Laufzeiten
- Anzahl Iterationen
- Backend-Verarbeitungszeit
- Engine-spezifische Timeouts

Buffer nicht gefunden

Bei beispielsweise:

```
SQL response was not found
```

prüfen, ob EXEC und BUFFER denselben Response-Namen verwenden.

```
- type: SQL
  op: EXEC
  response: demo_response

- type: SQL
  op: BUFFER
  response: demo_response
```

Condition-Wert existiert vor WHILE nicht

Wenn der Buffer erst im LOOP erzeugt wird:

Nicht:

```
mode: WHILE
```

sondern:

```
mode: DO_WHILE
```

verwenden.

---

25. Best Practices

DO_WHILE für Polling

Wenn der Status im LOOP gelesen wird:

```
mode: DO_WHILE
```

verwenden.

Technische Antworten in Buffer überführen

Bevorzugt:

```
ENGINE
-> BUFFER
-> CONDITION
```

Polling-Intervall sinnvoll wählen

Beispielsweise:

```
value: "1000"
```

```
value: "2000"
```

```
value: "4000"
```

abhängig vom Zielsystem.

Immer Safety Guards definieren

```
maxIterations: ...
timeoutMs: ...
```

---

26. Kurzreferenz für KI / RAG

```
AGATE LOOP ENGINE

TYPE:
LOOP

PURPOSE:
Repeated execution of nested AGATE steps.

REQUIRED:
mode
condition
maxIterations
```

```
timeoutMs
steps

MODE WHILE:
condition first
0..N executions

MODE DO_WHILE:
body first
1..N executions

RECOMMENDED FOR POLLING:
DO_WHILE

LOOP CONDITION SEMANTICS:

true
-> another iteration

false
-> successful LOOP exit

CONDITION LANGUAGE:
Use global AGATE Condition Handling.
Do not redefine Condition syntax inside LOOP.

PREFERRED PATTERN:

ENGINE EXEC
-> ENGINE BUFFER
-> WAIT
-> LOOP CONDITION

SAFETY:

maxIterations
timeoutMs

NO BREAK STEP.

DO_WHILE:
complete body executes before condition.

Therefore:
ENGINE
BUFFER
WAIT
CONDITION

means WAIT also executes in the final iteration.

AI GENERATION:

1. Prefer DO_WHILE for polling.
2. Generate maxIterations.
3. Generate timeoutMs.
4. Prefer ENGINE -> BUFFER -> CONDITION.
5. Keep LOOP conditions simple.
6. Refer to Condition Handling documentation for supported operators.
```